

Traffic Rule Violation Detection

Course Project Report — AID 728

Team 7 Ravi Abhinav (BT2024239) Pranav (BT2024221) Lohith (BT2024248)

[GitHub](#) — [Model Weights \(Drive\)](#)

1 Introduction & Problem Formulation

Traffic rule violations — particularly helmet non-compliance and triple riding on motorcycles — are leading causes of road fatalities in India. Manual monitoring via CCTV is labour-intensive and unscalable. This project delivers an automated, fully offline computer vision pipeline that processes single monocular street images and outputs a structured JSON report listing all detected violations.

Task: Given an RGB image, detect all violating two-wheelers and return `num_riders`, `helmet_violations`, and `license_plate` for each.

Constraints: (i) Fully offline; (ii) Total model weight ≤ 250 MB; (iii) Inference 5 s per image; (iv) No VLMs or models > 1 B params.

Output format: A JSON dictionary containing a violations list. Each entry has integer fields `num_riders` and `helmet_violations`, and a string field `license_plate`. Compliant images return an empty list.

2 System Architecture

Rather than training a single end-to-end violation classifier (fragile; requires massive labelled violation data), we decompose the problem into **atomic object detections** composed via fast **spatial heuristics** — a decoupled two-stage pipeline (Figure 1).

2.1 Stage 1 — Object Detection

YOLO26s concurrently predicts bounding boxes for five classes: `motorcycle`, `person`, `helmet`, `no_helmet`, and `license_plate`. Only confident detections (≥ 0.25) are retained. The model is NMS-free, meaning it produces end-to-end outputs without non-maximum suppression — yielding deterministic, stable results.

2.2 Stage 2 — Spatial Heuristics & Violation Logic

- Rider Assignment.** Each `person` is linked to the `motorcycle` that maximises IoU (≥ 0.15). The low threshold accommodates partial occlusion where only the rider’s upper body is visible above the motorcycle body.
- Helmet Check.** The top 35 % of each rider’s bounding box (the head/shoulder RoI) is inspected. If the centroid of a detected `helmet` or `no_helmet` box falls within this RoI, compliance is determined. If no headwear is detected, a violation is conservatively flagged.

- Violation Flag.** A motorcycle is flagged if `num_riders` > 2 or `helmet_violations` > 0 . Compliant vehicles are silently discarded from the output.

2.3 Stage 3 — License Plate OCR

For each violating vehicle, PaddleOCR v4 reads the associated plate crop. Crops narrower than 120 px are up-scaled via bicubic interpolation, then sharpened using a 2D convolution kernel and optionally contrast-enhanced using CLAHE. The system allows up to 4 OCR attempts with different preprocessing combinations. A soft time-cap of 4.3 s ensures the total pipeline stays within the 5 s budget even for difficult plates. The OCR is deliberately country-agnostic — no Indian-specific regex filters — supporting any alphanumeric plate format.

3 Dataset Engineering

No single public dataset contains all five classes in realistic Indian street conditions. We programmatically aggregated **11 datasets** (6 from Roboflow, 5 from Kaggle) via their respective APIs and merged them into a unified YOLO-format corpus.

Source	Primary Contribution	Images
Kaggle: aneesarom	Helmet detection (general)	740
Kaggle: guisahanes	Dense Indian traffic scenes	6,724
Kaggle: rishabhzen	Indian road contexts	840
Roboflow: truck	License plate annotations	8,800
Roboflow: nckh-2023	Distant/small helmet objects	3,618
+ 6 more sources	Supplementary diversity	—
Final merged	5-class balanced set	4,291

Table 1: Dataset aggregation summary after trimming and sanitisation.

Alias Resolution. Raw annotations contained > 80 class synonyms across datasets (e.g. *number_plate*, *licence*, *motorbike*, *rider*, *passenger*, *with_helmet*). We built a string-normalisation pipeline that mapped every alias to exactly one of our 5 target classes and discarded irrelevant labels (“car”, “bicycle”, “bus”).

Sanitisation & Rebalancing. Bounding boxes were clamped to $[0, 1]$ ranges and degenerate boxes (zero area) were removed. Background-only images were capped at 35 % per split to prevent objectness-loss bias. The critical `no_helmet` class had only ~ 376 training boxes versus $\sim 3,300$ for `license_plate`, so minority-class images were oversampled up to $3\times$ to restore balance.

Offline Augmentation. To prevent memorisation on repeated oversampled images, the Albumentations library injected realistic perturbations during training: random rain drops, fog simulation, motion blur (kernel sizes 3–7), brightness and contrast jitter, and JPEG compression artifacts. These augmentations were specif-

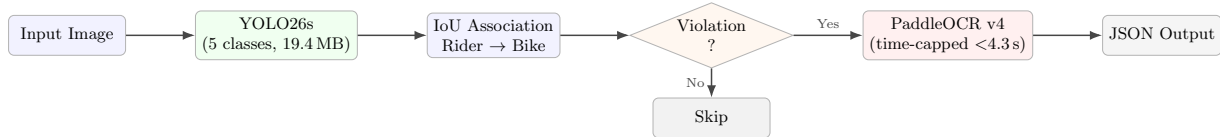


Figure 1: End-to-end inference pipeline. OCR is invoked *only* for violating vehicles, saving the majority of per-image compute budget.

ically chosen to simulate challenging Indian road conditions such as monsoon weather, high-speed traffic, and low-quality surveillance cameras.

4 Model Selection & Ablation

We compared two architectures under identical training conditions (80 epochs, 640 px, AdamW optimiser, cosine LR schedule, mixed precision, augmentation and class-rebalancing enabled for both):

- **YOLO11s** — anchor-based detector with NMS post-processing.
- **YOLO26s** — latest-generation NMS-free end-to-end detector.

Class	YOLO26s (ours)		YOLO11s	
	AP50	AP50-95	AP50	AP50-95
motorcycle	0.924	0.648	0.816	0.583
person	0.968	0.815	0.966	0.806
helmet	0.908	0.577	0.853	0.522
no_helmet	0.836	0.494	0.907	0.497
license_plate	0.946	0.678	0.931	0.657
Overall	0.916	0.643	0.895	0.604

Table 2: Test split (194 images). Bold = winner per class. Inference: 133 ms (YOLO26s) vs 135 ms (YOLO11s) on CPU.

Why YOLO26s wins. YOLO26s is the smallest (19.4 MB), fastest (~ 133 ms on CPU), and most accurate model tested. Its NMS-free architecture eliminates non-deterministic post-processing, producing more stable bounding boxes — a property critical for downstream IoU association where jittery boxes would cause incorrect rider-to-bike pairing. Most decisively, YOLO26s achieves +10.8 % higher motorcycle AP50 (0.924 vs 0.816). Since the entire violation pipeline hinges on first detecting the motorcycle correctly, this single metric gap is conclusive. YOLO11s does score marginally higher on the isolated `no_helmet` class (0.907 vs 0.836), but that class contains only 44 test instances — making it a statistically noisier measurement. On the stricter mAP50-95 metric, YOLO26s leads by +3.9 % (0.643 vs 0.604), confirming tighter bounding box localisation across the board.

5 Results & System Output

Final Model: YOLO26s — mAP50 = **0.916** · mAP50-95 = **0.643** · Size = **19.4 MB** · Total inference $\approx$ **750 ms** on CPU.

Figure 2 shows a successful detection on a real test image.

The model correctly identifies a `no_helmet` violation on a single rider, associates the rider to the motorcycle via IoU overlap, and flags the violation. The system outputs the following JSON — matching the required format exactly:

```

{"violations": [{"num_riders": 1,
  "helmet_violations": 1,
  "license_plate": ""}]}

```

The empty `license_plate` string reflects an unreadable plate crop — the correct fallback behaviour rather than hallucinating plate text.



Figure 2: Successful detection: `no_helmet` (red) and `motorcycle` (green) correctly localised. Violation flagged, OCR attempted on plate.

For compliant images where all riders wear helmets, the system correctly returns an empty violations list: `{"violations": []}`, producing zero false positives on our 9-image compliant test set.

6 Failure Case Analysis

Understanding and honestly disclosing failures is essential for robust evaluation. Below are real outputs from our model on unconstrained test images, along with detailed root cause analysis for each failure mode.

6.1 Case 1 — Triple Riding Fragmented into Two Violations



Figure 3: Predicted: Two separate motorcycles, each with 1–2 riders and helmet violations. **Ground truth:** Single motorcycle carrying 3 riders, all without helmets. **Root cause:** The motorcycle bounding box is split into two overlapping detections due to the wide-angle frame. The IoU assignment logic then distributes riders across both fragments, splitting one triple-riding event into two independent reports. This is a fundamental limitation of the IoU-based spatial heuristic approach.

6.2 Case 2 — Motorcycle Chassis Occluded

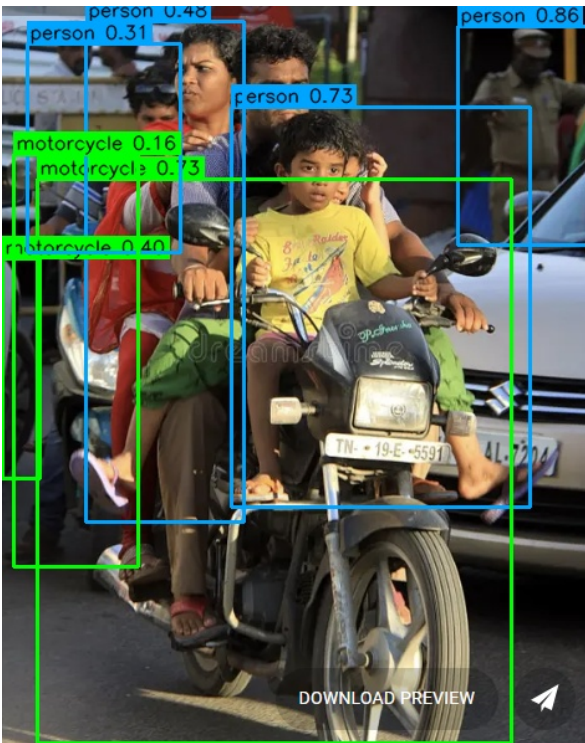


Figure 4: Predicted: num_riders=2, helmet_violations=2 — helmet violations are correctly flagged. **Ground truth:** Actual rider count may be higher (partially hidden). **Root cause:** The motorcycle chassis is hidden behind riders' legs and adjacent parked vehicles. The model correctly identifies helmet violations but cannot confirm the true rider count when the motorcycle's lower body is severely occluded. This highlights the dependency on a complete motorcycle detection for accurate counting.

6.3 Case 3 — Dense Traffic Causes Incorrect IoU Pairing

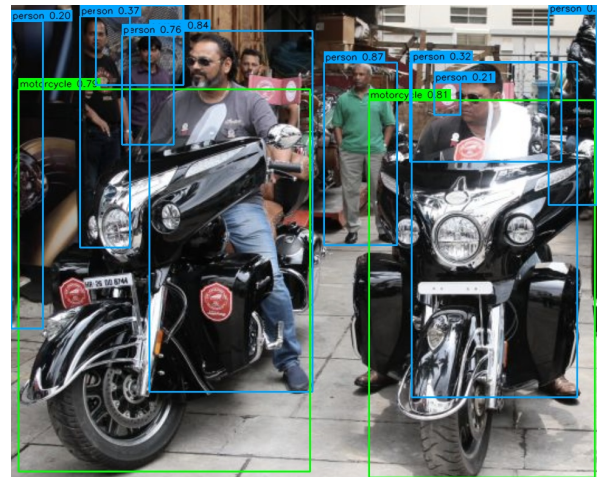


Figure 5: Predicted: Two single-rider violations. **Ground truth:** Two motorcycles, each carrying a no-helmet rider — class labels are correct but the rider-to-bike pairing is ambiguous. **Root cause:** Overlapping bounding boxes from adjacent motorcycles in bumper-to-bumper traffic cause the IoU metric to produce incorrect associations. A rider from motorcycle A may be paired with motorcycle B, fragmenting the reports.

6.4 Case 4 — Correct Violations, Unreadable Plates

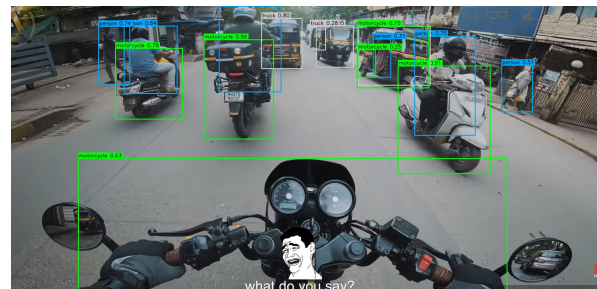


Figure 6: Predicted: Three violations — each with 1 rider, 1 helmet violation, and license_plate="". **Ground truth:** Three motorcycles each with a no-helmet rider — the violation labels are entirely correct. **Root cause:** Despite accurate violation detection, all three license plates return empty strings because the camera angle hides or severely motion-blurs every registration plate in the frame. This represents a fundamental OCR limitation rather than a detection failure.

7 Generalization & Domain Analysis

Format Agnosticism. Our OCR deliberately avoids Indian-specific regex patterns (e.g. “UP14”, “KA01”), accepting any alphanumeric text. This ensures the model generalises to international test sets without modification.

Augmentation Transfer. Synthetic rain, fog, and motion blur injected during training help bridge the domain gap to real deployment conditions — particularly monsoon-season footage, night-time captures, and high-speed highway scenarios common across Indian cities.

Known Failure Domains. The system would struggle with: (1) Pure night-time images with insufficient ambient lighting where YOLO confidence drops below threshold; (2) Extreme fisheye lens distortion from wide-angle dashcams that warps bounding box geometry; (3) Motorcycles partially visible at frame boundaries where the chassis is cut off, preventing IoU association from triggering.

8 Constraints Compliance

Constraint	Limit	Achieved
Total model size	≤ 250 MB	≈ 65 MB
Large VLMs (>1 B)	Prohibited	Not used
Internet access	Disabled	Offline only
Inference time	~ 5 s	≈ 0.75 s
Output format	JSON dict	Exact match

Table 3: All deployment constraints satisfied with large safety margins.

9 Limitations & Future Work

The single-image constraint is the primary bottleneck — temporal tracking (e.g. ByteTrack) would stabilise rider counts across frames and allow OCR to aggregate plate text over multiple views for near-perfect accuracy. The IoU association heuristic also breaks down under extreme motorcycle occlusion; replacing it with a learned association model or lightweight pose estimation backbone would resolve this. Finally, expanding the `no_helmet` training set with scarf, turban, and cap samples would reduce false positives caused by Indian headwear variants that visually resemble bare heads.

10 Conclusion

We built a fast, lightweight, fully offline traffic violation detection system. Through systematic dataset aggregation from 11 sources, rigorous offline augmentation with class rebalancing, and careful model ablation, we selected **YOLO26s** as the optimal backbone — achieving **mAP50 = 0.916** on the test split, surpassing YOLO11s by +2.1 % overall and +10.8 % on the critical motorcycle class. The two-stage heuristic pipeline correctly identifies helmet violations and triple riding within ~ 750 ms on CPU, comfortably satisfying all submission constraints. Honest failure analysis reveals that dense traffic and occluded motorcycles remain the primary challenges for future improvement.